$$
\begin{array}{rl}
e ::= x \mid c \mid \text{true} \mid \text{false} \mid k_{\#n} \mid () & \text{constants} \\
\quad \mid \textbf{let}\ x = e_1\ \textbf{in}\ e_2 & \text{let binding} \\
\quad \mid \textbf{uniform}(e_1, e_2) \mid \textbf{gaussian}(e_1, e_2) \mid \textbf{exponential}(e) \mid \ldots & \text{continuous distributions} \\
\quad \mid \textbf{discrete}(p_0, \ldots, p_{n-1}) & \text{discrete distribution} \\
\quad \mid e_1 < e_2 \mid e_1 \le e_2 \mid e_1 <_{\#n} e_2 \mid e_1 \le_{\#n} e_2 & \text{comparisons} \\
\quad \mid e_1 \,\&\&\, e_2 \mid e_1 \,||\, e_2 \mid \text{not } e & \text{Boolean operations} \\
\quad \mid \textbf{if}\ e_1\ \textbf{then}\ e_2\ \textbf{else}\ e_3 & \text{conditional} \\
\quad \mid (e_1, e_2) \mid \textbf{fst}\ e \mid \textbf{snd}\ e & \text{pairs} \\
\quad \mid \textbf{fun}\ x\ \to\ e \mid e_1\ e_2 \mid \textbf{fix}\ f\ x := e & \text{functions} \\
\quad \mid \textbf{observe}\ e \mid e_1; e_2 & \text{effects} \\
\quad \mid \text{nil} \mid e_1 :: e_2 \mid (\textbf{match}\ e\ \textbf{with} \mid \text{nil} \to e_{\text{nil}} \mid e_1 :: e_2 \to e_{\text{cons}}\ \textbf{end}) & \text{lists} \\
\quad \mid \textbf{diverge} & \text{diverge}
\end{array}
$$

Fig. 5. Syntax of the Slice language. Slice supports a variety of continuous distributions with one and two parameters; for a full list of supported distributions, see Appendix C.

We now turn to the more involved example in Figure 4, which Slice discretizes automatically. This example emphasizes how our discretization procedure operates across higher-order functions, unbounded recursion, and lists.

The program models the following process: We start by randomly sampling 4 intervals (l. 6 and 13), stored as a list of pairs, where the first component is a flag and the second component is a function that determines whether a given point falls within that interval. We then employ unbounded recursion (l. 36) to repeat the following a potentially unbounded number of times according to a geometric distribution: sample a point $y$ from a gaussian (l. 31), and update the flag of every interval to true whenever $y$ falls within that interval (l. 22). The program returns true iff *all* intervals have been hit at least once (l. 50 and 42). Exact inference on this program therefore determines the probability that this infinite stochastic process hits every interval at least once.

Slice *automatically* produces a *provably equivalent* (cf. §6) discrete program (shown in §A.5). Even though this program still contains unbounded recursion, our back-end automatically determines that its stochastic execution behavior can be modeled as a *finite-state* Markov chain. Our key insight is that exact inference can then be performed via *probabilistic model checking*: we use Storm [16], which automatically computes that the sought-after probability is 0.034474.... See §7.1 for details. To the best of our knowledge, our pipeline is the first capable of performing exact inference on suitable programs containing continuous sampling, higher-order functions, and unbounded recursion. In §7.1.2, we extend this program by conditioning *inside the recursion*.

## 3 The Slice Language and Type System

We now describe the language (§3.1), its types (§3.2), and the typing rules for discretization (§3.3). Henceforth, we use the terms *expression*, *subexpression*, and *program* somewhat interchangeably when the intended meaning is clear from context.

### 3.1 The Slice Language

The syntax in Figure 5 extends a standard higher-order functional core with probabilistic choices. Beyond constants, variables, comparisons, Boolean operators, conditionals, pairs, and lists, Slice offers first-class functions, recursion, and observations. The construct **observe**$(e)$ conditions on $e$ holding. Finite-domain constants $k_{\#n}$ range over $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$ as discussed below. For random sampling, we include the usual discrete distributions and any continuous distribution with a tractable cumulative distribution function (one- or two-argument variants such as exponential, laplace, uniform, gaussian, beta, gamma). Adding further distributions is straightforward as long as their CDFs are known; §C lists the ones implemented.

The expression **discrete**$(p_0, \ldots, p_{n-1})$ requires a nonempty probability vector: $n \ge 1$, every $p_i \ge 0$, and $\sum_i p_i = 1$. It returns $k_{\#n}$ with probability $p_k$.

These are ideal real-valued premises. The implementation separately validates finite binary64 weights. It normalizes compiler-generated weights only within a tight allowance for accumulated CDF arithmetic, and parses source vectors within a documented $10^{-4}$ rounded-input allowance. These engineering policies do not replace the exact premise above: binary64 evaluation and normalization do not by themselves establish exact preservation of ideal real-valued probabilities, which requires a separate numerical error analysis.

### 3.2 Types

We formalize the types that Slice supports. A *value* set $V$ is a finite set of reals or $\top$. Elements of $\{< x \mid x \in \mathbb{R}\} \cup \{\leq x \mid x \in \mathbb{R}\}$ are called *cuts*. Cut sets induce intervals partitioning the reals as follows: Given a non-empty cut set $B = \{\sim_1 x_1, \ldots, \sim_n x_n\}$ with $x_1 \leq \ldots \leq x_n$, we define the set Intervals($B$) of intervals it induces as expected, e.g., Intervals($\{<0.1, \leq 0.5\}$) = $\{(-\infty, 0.1), [0.1, 0.5], (0.5, \infty)\}$. The strict and non-strict cuts differ only in which side contains the boundary point: Intervals($\{<c\}$) = $\{(-\infty, c), [c, \infty)\}$, whereas Intervals($\{\leq c\}$) = $\{(-\infty, c], (c, \infty)\}$. Thus a boundary generated by $e < c$ places $c$ on the right, while a boundary generated by $e \leq c$ places $c$ on the left. Moreover, we let Intervals($\emptyset$) = $\{(-\infty, \infty)\}$ and Intervals($\top$) = $2^{\mathbb{R}}$. We denote by Intervals($B$)$_i$ the $i$-th interval of Intervals($B$), assuming Intervals($B$) is 0-indexed, i.e., $i \in \{0, ..., n\}$. We denote the set of all cut- and value sets, respectively, by $\mathcal{B}$ and $\mathcal{V}$.

*Definition 3.1 (Types).* For $n \in \mathbb{N}$, $B \in \mathcal{B}$, and $V \in \mathcal{V}$, types $\tau$ adhere to the grammar

$$\tau ::= \textbf{int} \mid \textbf{bool} \mid \textbf{unit} \mid \textbf{fin}(n) \mid \textbf{float}[B; V] \mid \tau_1 * \tau_2 \mid \tau_1 \to \tau_2 \mid \textbf{list}(\tau)\ .$$

Standard types cover integers, booleans, ordered pairs, functions, and lists. Finite types $\textbf{fin}(n)$ contain $\{0_{\#n}, \ldots, (n-1)_{\#n}\}$; the $\#n$-subscript reminds inference backends such as Dice that these values live in a finite domain. The float type $\textbf{float}[B; V]$ annotates a real-valued expression with its cut set $B$ (an over-approximation of the comparisons it participates in) and value set $V$ (an over-approximation of the values it can take). $B = \top$ means the expression may be compared to arbitrary reals; $B = \emptyset$ means it will never be compared; $V = \top$ indicates an unrestricted result.

### 3.3 Typing Rules

The typing rules are split into three parts: Rules specific to float types and continuous distributions (Figure 6), standard rules for all other language constructs (Figure 7), and subtyping (Figure 8).

To understand the float typing rules, it is helpful to understand how discretization works. A type **float**[B; V] is discretized as follows: If $B \neq \top$ is a finite set of cuts, then the type is discretized to a type $\textbf{fin}(n+1)$ where $n+1 = |\text{Intervals}(B)|$ is the number of intervals induced by $B$. For instance, if $B = \{<0.5, \leq 0.8\}$, then the type is discretized to $\textbf{fin}(3)$, where $0_{\#3}$ represents the interval $(-\infty, 0.5)$, $1_{\#3}$ represents the interval $[0.5, 0.8]$, and $2_{\#3}$ represents the interval $(0.8, \infty)$. If $B = \top$, then no discretization is performed, and the type remains **float**.

To understand the typing rules, keep in mind that the cut sets $B$ need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

*Constants.* Rule Float-Const assigns a constant $c$ the type $\textbf{float}[B; V]$ whenever has($V, c$) holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in $V$ ensures later comparisons know which values must remain distinguishable.

*Continuous distributions.* Rule Continuous-1 types a single-argument distribution when its argument $e$ has type $\textbf{float}[B; V]$ and recovers($B, V$):

*Definition 3.2.* Let $B \in \mathcal{B}$ and $V \in \mathcal{V}$. We say that recovers($B, V$) if $B = \top$ or ($V \neq \top$ and every interval $I$ induced by $B$ contains at most one value in $V$, i.e., $\forall I \in \text{Intervals}(B) : |V \cap I| \leq 1$).

Intuitively, recovers($B, V$) means that the discretization induced by $B$ is lossless on $V$: from the resulting interval index, the original value in $V$ can be uniquely recovered. For example, $\textbf{exponential}(e)$ can be discretized only when $e$ takes finitely many recoverable values. Thus $V$ must be finite and $B$ must separate those values so the discretized program can determine which one occurred. §5 revisits this invariant. The rule for two-argument distributions is analogous.

*(1) Probability measure.* We show for each $x \in X$, the map

$$A \mapsto f(x)(A)$$

is a probability measure.

Fix any $x \in X$. Then $f(x)$ is the zero measure: it satisfies $\sigma$-additivity since

$$f(x)\left(\bigcup_i A_i\right) = 0 = \sum_i 0 = \sum_i f(x)(A_i)$$

for every pairwise disjoint family $(A_i)_i$, and total mass $f(x)(Y) = 0 \leq 1$.

*(2) Measurable function.* For any $A \in \mathcal{F}_Y$, the map $x \mapsto f(x)(A) = 0$ is the constant-zero function on $X$, hence measurable.

Thus both conditions hold, so $f$ is a Markov kernel. □

Lemma F.33 (Composition of Markov kernels). *Let $(X, \Sigma_X)$, $(Y, \Sigma_Y)$, and $(Z, \Sigma_Z)$ be measurable spaces. If*

$$P : X \to \mathcal{D}(Y) \quad \text{and} \quad Q : Y \to \mathcal{D}(Z)$$

*are Markov kernels, then their composition*

$$(P \gg\!= Q)(A) \;:=\; \int_{x \in supp(P)} Q(x)(A)\, P(dx)$$

*defines a Markov kernel*

$$P \gg\!= Q : X \to \mathcal{D}(Z).$$

Proof. See [30, Lemma 1.38]. □

### F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ a well-typed expression of type $\tau$ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression $e : \tau = \textbf{uniform}(e_1 : \textbf{float}[B_1; V_1], e_2 : \textbf{float}[B_2; V_2]) : \textbf{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \neq \top$, we introduce an auxiliary statement $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ and let

$$\mathcal{D}[\![e : \tau]\!] = \mathtt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![e_1 : \textbf{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \textbf{float}[B_2; V_2]]\!])\,.$$

We define the semantics of $\mathtt{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the let-bindings and the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule :

$$\frac{e_1 : \textbf{fin}(|B_1| + 1) \qquad e_2 : \textbf{fin}(|B_2| + 1) \qquad \text{recovers}(B_1, V_1) \qquad \text{recovers}(B_2, V_2)}{\mathtt{DC}_{B,B_1,B_2,V_1,V_2}(e_1, e_2) : \textbf{fin}(|B| + 1)}\ \text{TDiscreteCase}$$

Equipped with these notions, we define the small-step semantics of our language:

*Definition F.34.* The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \bot)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

In the Discrete row, the typing premises ensure that the displayed weights form a nonempty probability vector. Hence $\text{Discrete}(p_0, \ldots, p_{n-1}) = \sum_{i=0}^{n-1} p_i\, \delta_{i \# n}$ is a probability measure in $\mathcal{D}(Expr_{\textbf{fin}(n)} + \bot)$. The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\textbf{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\bot)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace $\sigma$-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$

*and let $B \subseteq \mathbb{R}^d$ be equipped with the subspace $\sigma$-algebra*

$$\mathcal{F}_B := \{B \cap N \mid N \in \mathcal{B}(\mathbb{R}^d)\}.$$